



Nerdplexity LLM Observability and Hallucination Risk Dashboard

Executive summary

Enabled connectors: GitHub ¹ . ²

Your repo already logs per-turn events locally (IndexedDB) and already ships an analytics surface (“PromptOps”) with metrics, event tables, risk bands, and cost estimates. The fastest path to a research-backed “Nerdplexity” observability and hallucination dashboard is an additive extension of the existing PromptEvent logging pipeline, not a rewrite. ³ ⁴ ⁵ ⁶

Two architecture facts drive the design:

First, the README advertises `/promptscope`, and the codebase already contains a `PromptScopeRoute.tsx` that lazy-loads the dashboard behind a `PROMPTOPS_ENABLED` localStorage flag. The main router currently exposes `/promptops`, `/dashboard`, `/events`, `/benchmark`, and conditionally shows a “PromptOps” link only on `/`. Nerdplexity should add a route alias for `/promptscope` without removing anything. ² ⁷ ⁸

Second, your current “hallucination risk” and “groundedness” are heuristic composites (judge score, user rating, context bloat, over-generation, and “web enabled + citations present”). A research-backed dashboard needs a user-facing taxonomy and explainable signals tied to evidence support, truncation/stop reasons, and cross-run consistency, consistent with the literature on faithfulness/factuality and black-box hallucination detection. ⁹ ⁶ ¹⁰

Key deliverables in this design:

A normalized TraceEvent schema that maps provider stop/usage fields into a single structure and stores everything local-first (IndexedDB). Metrics that focus on faithfulness to provided context, unverified claim density, prompt ambiguity, stop-reason integrity, and multi-sample consistency, with logprobs treated as internal-only heuristics. A dashboard UX that explains “why risky” for each message and guides prompt edits and verification behavior.

Repo telemetry baseline and integration points

Local-first storage and existing event model

The app already uses IndexedDB (via Dexie) as the primary persistence layer for conversations, settings, and analytics events. The `ChatDatabase` schema includes an `events` table keyed by `{id, ts,`

`corr_id, provider, model}` and the app migrates from `localStorage` into `IndexedDB` when older keys exist. ¹¹

The current `PromptOps` logger writes `PromptEvent` objects into that same `db.events` table, generating a synthetic event id and ordering by timestamp. ⁵

The “ADD-ONLY” instrumentation pattern already exists

The `PromptOps` instrumentation wrapper records latency and “TTFT” timestamps via `performance.now()` and logs an event with `provider`, `model`, `settings`, `business`, plus `usage`, `safety`, `quality`, `prompt`, `retrieval`, `answer` when present. ⁴

Important caveat for Nerdplexity: in the current chat flow, `markFirstToken()` is invoked after the full JSON response arrives, not on the first streamed token. That makes TTFT behave like “time to response received,” not time-to-first-token. Nerdplexity should keep the field but label it accurately unless true streaming gets implemented. ³ ⁴

UI entry points and the `/promptscope` mismatch

The README instructs users to visit `/promptscope` for analytics. ²

In the router (`App.tsx`), analytics routes exist as `/promptops` (landing), `/dashboard` (metrics), `/events`, and `/benchmark`. The “PromptOps” link appears only on the home route and is gated by `PROMPTOPS_ENABLED`. ⁷

A dedicated `PromptScopeRoute.tsx` exists and lazy-loads a dashboard behind the same gate. It currently is not registered in the main router. Nerdplexity should add `Route path="/promptscope"` to mount `PromptScopeRoute` and preserve existing routes unchanged. ⁸ ⁷

Provider routing and request enrichment

Server-side, `/v1/chat` routes requests to providers (`openai`, `anthropic`, `deepseek`, `gemini`, `local-ollama`) and supports `auto` provider detection based on model name patterns. It also supports optional web search augmentation (`web_search`) and a “show reasoning” mode that injects a `<thinking>` block request and then strips that block into a separate response field. ¹²

This matters for a hallucination dashboard because it creates three distinct “context regimes” per turn:

No external context (pure parametric response). Web-augmented context (retrieved snippets injected into prompt, plus `webSearchResults` returned to the client). Reasoning requested (the model outputs extra tokens that are not part of the final answer text).

Nerdplexity should explicitly track which regime occurred per turn, then compute different reliability metrics per regime.

Security and key handling constraints you already satisfy

The settings modal states that keys stay local in the browser and are never sent anywhere except directly to providers. Keys persist in IndexedDB settings. Nerdplexity should keep this posture and store analytics local-first by default. ¹³ ¹¹

Provider response signals and normalized TraceEvent schema

This section covers:

What reliable per-request/per-response fields exist across providers. Which “confidence-like” fields exist, and why Nerdplexity treats them as internal-only heuristics. A concrete TraceEvent schema and mapping table.

Provider signals you can reliably collect

OpenAI-style Chat Completions API

The OpenAI Chat Completions response contains `choices[].finish_reason` with enumerated reasons like `stop`, `length`, `content_filter`, `tool_calls`, plus optional `logprobs` fields when requested. The response includes `usage.prompt_tokens`, `usage.completion_tokens`, `usage.total_tokens` and token detail breakdown objects. ¹⁴

Anthropic Messages API

Anthropic includes a `stop_reason` field in every successful messages response, and `usage.input_tokens` and `usage.output_tokens`. Anthropic’s docs emphasize that `stop_reason` is critical for robust handling and list reasons including `end_turn`, `max_tokens`, and `stop_sequence`. ¹⁵

Gemini GenerateContent response model

Gemini exposes `usageMetadata` token counts, `finishReason` per candidate, safety ratings, `citationMetadata`, and grounding structures (`groundingMetadata`, `groundingChunks`, `groundingSupports`) when grounding is enabled. Gemini also exposes `avgLogprobs` and token log-likelihood structures. ¹⁶

DeepSeek Chat Completions API

DeepSeek documents OpenAI-compatible `finish_reason`, streaming with `include_usage`, a required `system_fingerprint`, and a usage object with cache hit/miss token counts (`prompt_cache_hit_tokens`, `prompt_cache_miss_tokens`) plus reasoning token breakdowns. ¹⁷

Ollama local API

Ollama exposes timing and token evaluation metrics such as `total_duration`, `load_duration`, `prompt_eval_count`, `prompt_eval_duration`, `eval_count`, and `eval_duration`, measured in nanoseconds. Usage fields arrive as part of the final streaming chunk when streaming is enabled. ¹⁸

Current provider adapter gaps in your repo

Your provider adapters return the assistant text plus `usage`, but they do not return stop reasons or provider-specific timing fields to the frontend today. ¹⁹ ²⁰ ²¹ ²²

The local Ollama adapter aggregates streamed JSON lines into a final string and returns `usage: null` and `raw: null` in the streaming branch, which blocks Nerdplexity from leveraging Ollama's native timing fields. Nerdplexity Phase 1 should pass through the final-chunk usage/timing metrics. ²³ ¹⁸

Treating logprobs as internal-only heuristics

OpenAI-style APIs and Gemini/DeepSeek expose token log-likelihood fields, but these are not comparable across providers and are hard to interpret for end users. Nerdplexity should store a logprobs summary under `internal.*` for debugging and internal model diagnostics, but should not display it as user-facing "confidence." ²⁴

Normalized TraceEvent schema

Nerdplexity should add a normalized TraceEvent that is stored alongside existing PromptEvent entries, without schema breakage. Make it additive: older events lack the new fields, and dashboards handle nulls.

TraceEvent (conceptual JSON schema)

```
{
  "id": "evt_...",
  "ts": "2026-02-12T20:15:00.000Z",
  "corr_id": "uuid",
  "session_id": "conversation_id",
  "span": { "name": "chat.turn", "kind": "llm" },

  "provider": {
    "name": "openai|anthropic|gemini|deepseek|local-ollama",
    "model": "string",
    "response_id": "string|null",
    "system_fingerprint": "string|null",
    "model_version": "string|null"
  },

  "request": {
    "params": {
      "temperature": 0.7,
      "max_tokens": 4000,
```

```

    "top_p": null,
    "stream": false
  },
  "features": {
    "web_search": false,
    "show_reasoning": false
  },
  "prompt_stats": {
    "prompt_chars": 1234,
    "messages_count": 9
  }
},

"timing": {
  "latency_ms": 1842,
  "ttft_ms": null,
  "provider_total_duration_ns": null,
  "provider_load_duration_ns": null,
  "provider_prompt_eval_duration_ns": null,
  "provider_eval_duration_ns": null
},

"usage": {
  "prompt_tokens": 932,
  "completion_tokens": 381,
  "total_tokens": 1313,
  "cache_hit_tokens": null,
  "cache_miss_tokens": null,
  "reasoning_tokens": null
},

"generation": {
  "stop_reason_norm": "stop|length|max_tokens|tool_calls|refusal|safety|
other",
  "stop_reason_raw": "provider-specific string|enum"
},

"retrieval": {
  "mode": "none|web",
  "sources_count": 0,
  "sources": []
},

"answer": {
  "text": "string",
  "citations": ["url or id"],
  "has_thinking": false
},

```

```

"metrics": {
  "risk_score": 0.0,
  "risk_band": "Low|Medium|High",
  "prompt_ambiguity": 0.0,
  "unverified_claim_rate": 0.0,
  "context_faithfulness": null,
  "self_consistency": null,
  "stop_reason_risk": 0.0
},

"internal": {
  "logprobs_summary": null,
  "raw_provider_hash": "sha256:..."
}
}

```

Provider fields to TraceEvent mapping table

Plain-text table (provider → fields → TraceEvent mapping → UI surface)

Provider	Provider response fields	TraceEvent mapping	User-facing vs internal	
				OpenAI-style
	choices[].finish_reason, usage., logprobs	generation.stop_reason_raw/norm, usage.,	Show stop reason + usage.	
	(Chat Completions)	(optional)	internal.logprobs_summary (optional)	Hide logprobs.
14				
	Anthropic Messages	stop_reason, stop_sequence, usage.		
	generation.stop_reason_raw/norm,	Show stop reason + usage.		
	No logprobs.			
15				
	Gemini Generate	usageMetadata, finishReason, safetyRatings,		
	generation.stop_reason_raw/norm,	Show stop reason + safety +		
	Content	citationMetadata,		
	groundingMetadata,	attribution in retrieval/answer,		
	citations/grounding.			
	avgLogprobs/logprobsResult			
	internal.logprobs_summary (optional)	Hide avgLogprobs/logprobs.		
16				
	DeepSeek			
	finish_reason, system_fingerprint, usage	generation.stop_reason_raw/norm, usage.,	Show stop reason +	
	caching +			
	incl. cache hit/miss, reasoning_tokens	provider.system_fingerprint, usage.cache_		
	usage.			
	Hide logprobs.			
17				
	Ollama local	done_reason, total/load durations,		
	generation.stop_reason_raw/norm,			
	Show durations + throughput.			
	prompt_eval_count/duration, eval_count/dur			
	timing.provider_ns,			
	usage.local token counts	Hide logprobs.		
18				

Hallucination taxonomy and research-backed metric designs

Product definition of “hallucination” for Nerdplexity

Use a product definition that maps to what users experience:

A hallucination is a claim presented as factual that lacks support from available evidence (provided context or retrieved sources), conflicts with that evidence, or exhibits instability across re-sampling runs.

This aligns with the faithfulness/factuality framing in summarization research (hallucinations as unfaithful content) and with sampling-based black-box detection for LLM factuality. ¹⁰

User-facing categories and how they show up

Unsupported-by-context claims User pastes notes, asks for a summary, and the model adds extra dates not in the notes. Maynez et al. report that abstractive systems often hallucinate content unfaithful to the source, and entailment-style measures correlate better with faithfulness than classic surface metrics. ²⁵

Overconfident factual claims without evidence User asks “What is the price and contract term for vendor X?” The model outputs numbers with no citations and no “as-of” date. The Ji et al. survey frames this as a reliability issue across NLG tasks and highlights measurement/mitigation as a broad research area. ²⁶

Inconsistent answers across multiple runs User asks a technical question, the model alternates between two incompatible statements. SelfCheckGPT’s core claim: sampled outputs diverge more for hallucinated facts and remain more consistent for grounded facts in black-box settings. ²⁷

Guessing under prompt ambiguity User asks “When is the deadline?” with no course, timezone, or doc attached. The model invents a date. The Ji et al. survey treats ambiguity and missing grounding as a recurring driver of hallucination behavior across tasks. ²⁶

Concrete metrics Nerdplexity should compute per interaction

All metrics below fit your constraints: preserve current app flow, store local-first, and treat logprobs as internal-only.

Prompt ambiguity score (PAS)

Goal: flag prompts that invite guessing.

Computation (deterministic, local) Extract features from the user prompt and conversation state:

Missing anchors: no “as-of date,” no geography, no dataset/source reference, no entity disambiguation. Unbounded comparatives: “best,” “top,” “most,” without criteria. Undefined deliverable: no format, length, or verification requirement.

$PAS = \text{sigmoid}(\text{weighted_sum}(\text{features}))$.

Inputs: prompt text, settings flags (web_search, reasoning), optional mode. ³ ¹¹

Cost/compute: $O(n)$ string scanning, no model calls.

UI: Prompt Coach checklist and rewrite suggestions.

Unverified claim rate (UCR)

Goal: estimate how many answer statements lack evidence pointers.

Computation (local first-pass) Split answer into sentences. Mark a sentence as “claim-like” if it contains numbers/dates, named entities, or strong factual verbs (“is,” “was,” “released,” “costs,” “located”). Mark a claim as “verified” if one of these holds:

It references a citation marker from retrieved sources. It contains an explicit uncertainty qualifier (“not enough info,” “unknown,” “needs verification”). It directly quotes provided context (for pasted text workflows).

$$UCR = (\# \text{ claim-like sentences without verification}) / (\# \text{ claim-like sentences}).$$

Inputs: answer.text, answer.citations, retrieval.sources. Nerdplexity must start logging answer text into PromptEvent/TraceEvent since current turn logs store prompt_chars but do not store answer text. 3 4

Cost/compute: low. Regex + sentence split.

UI: Answer Audit highlight overlay and “claims needing support” list.

Context faithfulness score (CFS)

Goal: check whether claims are supported by the provided context when context exists.

Research basis: Maynez et al. report that entailment measures correlate better with faithfulness than standard metrics, motivating an entailment-style or support-check approach. 25

Computation (phased) Phase 2 heuristic: For each claim sentence, retrieve top-k matching context spans from webSearchResults snippets and from any “context injected” blocks. Score support using lexical overlap, quote coverage, and citation linkage.

Phase 3 judge-based: Run an NLI/entailment check: context span $\Rightarrow$ claim (entailed / contradicted / unknown). CFS = mean entailment score.

Inputs: answer text, retrieval source texts/snippets, optional local judge model for NLI.

Cost/compute: Heuristic: low. Judge-based: medium-high, depends on judge model and number of claims.

UI: Per-claim “Supported / Unsupported / Unknown” badges with expandable evidence.

Self-consistency score (SCS)

Goal: detect instability as a risk signal.

Research basis: SelfCheckGPT proposes sampling-based cross-checking without external databases; divergence across samples signals hallucination risk. ²⁷

Computation (opt-in due to cost) On user click "Stability check," re-run the same prompt N times (N=3 default) at temperature > 0, then compare claim sets across runs.

SCS = 1 - claim_variance. Also expose "conflict list" bulleted by claim.

Inputs: multiple runs' answer texts.

Cost/compute: high relative to single run because it triggers extra provider calls.

UI: Stability panel on a per-turn basis plus a compare view.

Stop-reason integrity signal (SRI)

Goal: treat truncation and tool call stops as reliability hazards.

Computation Normalize provider stop reasons into:

Natural completion (stop/end_turn/STOP) → low risk Truncation (length/MAX_TOKENS/max_tokens) → medium risk + “possibly cut off” Safety/refusal → separate “policy stop” bucket, not hallucination Tool call stops → “incomplete until tool results consumed”

Inputs: provider stop_reason/finish_reason. Sources: OpenAI finish_reason in chat completions. ¹⁴
Anthropic stop_reason. ¹⁵ Gemini finishReason. ¹⁶ DeepSeek finish_reason and include_usage
streaming behavior. ¹⁷ Ollama done_reason and timing fields. ¹⁸

Cost/compute: trivial string mapping.

UI: Timeline badges and “Truncated response” banner.

Metrics summary table (computation, inputs, cost, UI)

	Metric	Computation	Inputs	Cost/compute	UI element	
ambiguity score	weighted feature model over prompt	user prompt, settings	low	Prompt Coach + rewrite diff	Unverified claim rate	claim sentence detection + evidence answer text, citations, sources
low	claim highlights	Context faithfulness	claim→support spans→(heuristic/NLI)	answer + context snippets	low→medium/high	supported/unsupported heatmap
Self-consistency	N re-samples + claim variance	multiple answers	high (extra calls)	Stability score + conflicts	Stop-reason integrity	normalize stop reason
finish_reason/stop_reason	low	banners + timeline badges	Cost/latency efficiency	token + latency aggregations	usage + timing	low
Cost & Perf (existing)						

```
+-----+-----+-----+-----+
+-----+
```

Dashboard UX design with wireframes and data flow

Nerdplexity dashboard layout

Keep existing PromptOps pages intact. Add one new dashboard surface focused on reliability, plus a route alias:

Route alias: `/promptscope` → existing PromptOps landing or to the new Nerdplexity Reliability page (your choice). Preserve `/promptops`, `/dashboard`, `/events`, `/benchmark`. 7 2

New page name suggestion: "Reliability Dashboard" under PromptOps nav.

ASCII wireframe

```
+-----+
+
| Nerdplexity Reliability Dashboard      Session: <conversation>  Provider/
Model |
| Tabs: Timeline | Prompt Coach | Answer Audit | Compare | Cost &
Perf  |
+-----+
+

TIMELINE
Turn  Message preview                Risk   Drivers
Actions
----  -----
-----
#18   "Summarize the PDF..."         HIGH   UCR↑ CFS↓ SRI(truncated)      View
audit
#17   "What is the deadline?"         HIGH   PAS↑ UCR↑
Rewrite prompt
#16   "Cite sources for X..."        LOW    Grounded citations present
Compare models

PROMPT COACH
- Ambiguity checklist (missing date, scope, constraints)
- Suggested rewrite (minimal diff)
- Estimate: token/cost impact

ANSWER AUDIT
- Claim list:
  [!] Claim 1 (no evidence)      Suggested fix: enable web, attach source, ask
```

```

follow-up
  [✓] Claim 2 (supported)      Evidence: source #2 snippet
- Stop reason banner: "MAX_TOKENS: answer likely incomplete"
- Button: Run stability check (3 samples)

COMPARE
- Same prompt across providers/models
- Agreement score, conflict list
- Side-by-side: cost, latency, stop reasons

COST & PERF
- Keep existing charts (latency p50/p95, tokens/day, cost/100) and add cache insights

```

Mermaid data flow diagram (local-first)

```

flowchart LR
  UI[Chat UI] -->|POST /v1/chat| S[Server router]
  S --> P[Provider adapters]
  P --> S --> UI

  UI -->|startTurn/endOk/endError| E[Event logger]
  E --> IDB[(IndexedDB: events table)]

  IDB --> SC[Local scoring pipeline]
  SC --> D[Dashboard views]

```

This matches your current “chat → /v1/chat → provider → log to IndexedDB → dashboards read events” structure. 3 12 5

Key UX guardrail

Every “risk” surface must show drivers and evidence links. Do not show a single opaque score. This aligns with survey guidance that hallucination detection depends on task and context and resists a universal “truth score.” 26

Implementation roadmap, personas, and failure modes

Phased implementation roadmap

Each phase preserves existing app functionality. Treat all changes as additive. Store analytics local-first by default. 11

Phase 1 focus: expand TraceEvent logging inputs

Add TraceEvent fields without breaking existing PromptEvent dashboards. Log answer text and citations in `turn.endOk` (the dashboard needs answer text for claim metrics). Current turn logging includes `prompt_chars` and `usage`, but not answer text. ³ ⁴ Return and log stop reasons: OpenAI: `choices[0].finish_reason`. ¹⁴ ¹⁹ Anthropic: `stop_reason`. ¹⁵ ²⁰ Gemini: `finishReason`, `promptFeedback`, `usageMetadata`. ¹⁶ ²¹ DeepSeek: `finish_reason`, `system_fingerprint`, cache hit/miss usage, streaming include_usage shape. ¹⁷ ²² Ollama: pass through timing fields (`total_duration`, `eval_duration`, etc.) and `done_reason`. ¹⁸ ²³ Normalize “context regime” flags from chat settings: `web_search`, `show_reasoning`. ³

Phase 2 focus: ship explainable, low-cost reliability scoring

Implement PAS, UCR, and SRI locally with deterministic rules. Replace the current hallucination heuristic label with “Legacy risk” and introduce “Evidence risk” (PAS/UCR/SRI). Keep the legacy score available for backward compatibility. ⁹ Add a new “Answer Audit” view driven by answer text + citations.

Phase 3 focus: research-grade checks and comparisons

Add opt-in SelfCheck-style stability check with N=3 sampling, based on SelfCheckGPT’s sampling divergence framing. ²⁷ Add context faithfulness heuristics first, then an optional entailment judge (local model or a user-selected judge key), consistent with Maynez et al.’s finding that entailment correlates better with faithfulness. ²⁵ Add “Compare providers/models” view and compute agreement and stop reason deltas.

Mermaid timeline chart

```
gantt
  title Nerdplexity implementation timeline
  dateFormat YYYY-MM-DD

  section Phase 1
    TraceEvent schema + answer logging           :a1, 2026-02-13, 14d
    Stop-reason + provider usage normalization:a2, 2026-02-13, 14d
    Ollama timing passthrough                     :a3, 2026-02-20, 10d
    /promptscope route alias                     :a4, 2026-02-13, 3d

  section Phase 2
    Prompt ambiguity + claim rate scoring         :b1, 2026-02-27, 14d
    Timeline + Answer Audit UX                   :b2, 2026-02-27, 14d

  section Phase 3
    Stability check (SelfCheck-style)             :c1, 2026-03-13, 21d
    Faithfulness (heuristic → entailment)         :c2, 2026-03-20, 28d
    Model/provider compare + agreement            :c3, 2026-03-27, 21d
```

Persona-driven success criteria

Student

Primary risk: summaries that insert unsupported facts when the student expects strict faithfulness to provided notes. Maynez et al. show hallucinated unfaithful content is common in abstractive generation tasks. ²⁵

Dashboard needs: Answer Audit with “supported by notes/snippets” flags. Prompt Coach reminders to paste sources and request citations.

Success after a month: Lower UCR on study outputs. Fewer re-asks driven by invented details. More consistent use of “use only the provided notes” prompts.

Founder

Primary risk: confident market claims with no evidence or stale context.

Dashboard needs: UCR + PAS surfaced prominently. Stop-reason banners to catch truncated “half answers.” Compare view across providers to expose unstable claims.

Success after a month: More answers with explicit citations and “as-of” anchors. Lower rate of “numbers with no source” outputs.

Engineer

Primary risk: incorrect implementation details and inconsistent edge-case guidance.

Dashboard needs: Stability checks aligned with SelfCheckGPT (look for claim divergence). ²⁷ Stop reason integrity badges (truncation and tool-call stops). Cost/latency view that includes caching fields (DeepSeek cache hit/miss). ¹⁷

Success after a month: Lower frequency of production-impacting wrong instructions. Higher reuse of stable prompt templates with clear constraints.

Failure modes and UI guardrails

Creative tasks and opinion prompts Risk scoring should downshift automatically when the user selects a “creative” intent, since factuality is not the evaluation goal. Use the Ji et al. survey framing: hallucination definitions vary by task and evaluation target. ²⁶

Stable but wrong answers Self-consistency is not correctness. A model repeats the same wrong claim across samples. Nerdplexity should label SCS as “stability,” not “truth,” consistent with SelfCheckGPT’s purpose as a black-box consistency signal. ²⁷

Stop reasons as reliability hazards, not hallucination proof A truncation stop (`length`, `MAX_TOKENS`, `max_tokens`) signals incompleteness, not hallucination. A safety stop signals policy constraints. The UI should separate “completion integrity” from “evidence support.” ²⁸

What Nerdplexity adds relative to a standard chat app

A standard chat UI shows the answer and maybe token counts. Nerdplexity adds a local-first trace per interaction that (1) normalizes provider stop/usage signals, (2) decomposes “hallucination risk” into explainable categories tied to evidence and consistency, and (3) provides prompt rewrites and verification workflows driven by claim-level analysis and provider stop reasons, grounded in faithfulness research and black-box consistency methods. ²⁹

⁷ **packages/web/src/App.tsx**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/App.tsx>

² **README.md**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/README.md>

³ **packages/web/src/components/Chat.tsx**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/components/Chat.tsx>

⁴ **packages/web/src/promptops/instrument.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/promptops/instrument.ts>

⁵ **packages/web/src/promptops/logger.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/promptops/logger.ts>

⁶ **packages/web/src/promptops/metrics.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/promptops/metrics.ts>

⁸ **packages/web/src/routes/PromptScopeRoute.tsx**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/routes/PromptScopeRoute.tsx>

⁹ **packages/web/src/promptops/hallucination.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/promptops/hallucination.ts>

¹⁰ ²⁵ ²⁹ **<https://aclanthology.org/2020.acl-main.173/>**

<https://aclanthology.org/2020.acl-main.173/>

¹¹ **packages/web/src/lib/db.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/lib/db.ts>

¹² **packages/server/src/server.ts**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/server.ts>

¹³ **packages/web/src/components/SettingsModal.tsx**

<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/web/src/components/SettingsModal.tsx>

- 14 24 28 <https://platform.openai.com/docs/api-reference/chat/create-chat-completion>
<https://platform.openai.com/docs/api-reference/chat/create-chat-completion>
- 15 <https://docs.anthropic.com/en/api/handling-stop-reasons>
<https://docs.anthropic.com/en/api/handling-stop-reasons>
- 16 <https://ai.google.dev/api/rest/generativelanguage/CitationMetadata>
<https://ai.google.dev/api/rest/generativelanguage/CitationMetadata>
- 17 <https://api-docs.deepseek.com/api/create-chat-completion>
<https://api-docs.deepseek.com/api/create-chat-completion>
- 18 <https://docs.ollama.com/api/usage>
<https://docs.ollama.com/api/usage>
- 19 [packages/server/src/providers/openai.ts](#)
<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/providers/openai.ts>
- 20 [packages/server/src/providers/anthropic.ts](#)
<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/providers/anthropic.ts>
- 21 [packages/server/src/providers/gemini.ts](#)
<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/providers/gemini.ts>
- 22 [packages/server/src/providers/deepseek.ts](#)
<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/providers/deepseek.ts>
- 23 [packages/server/src/providers/local-ollama.ts](#)
<https://github.com/sanketmuchhala/Project-Green-Lantern/blob/701822abc6e0c357d036814b9f95b0e6d7d77c3c/packages/server/src/providers/local-ollama.ts>
- 26 <https://aievidencehub.org/lib/5RYUSHFD>
<https://aievidencehub.org/lib/5RYUSHFD>
- 27 <https://aclanthology.org/2023.emnlp-main.557/>
<https://aclanthology.org/2023.emnlp-main.557/>